

Vectorising the DHN-NCE loss function

Zain Nomani

October 15, 2025

Abstract

This document summarizes my process of reconstructing and analyzing the loss function proposed in a healthcare AI research paper. The goal is to understand the underlying implementation, benchmark performance, and identify potential improvements.

0.1 Loss Function

The loss function was defined as:

$$\begin{aligned}\mathcal{L} &= \mathcal{L}^{v \rightarrow t} + \mathcal{L}^{t \rightarrow v} \\ \mathcal{L}^{v \rightarrow t} &= - \sum_{i=1}^B \frac{I_{pi}^T T_{pi}}{\tau} + \sum_{i=1}^B \log \sum_{j \neq i} \exp\left(\frac{I_{pi}^T T_{pj}}{\tau}\right) \mathcal{W}_{I_{pi} T_{pj}}^{v \rightarrow t} \\ \mathcal{W} &= (B-1) \frac{\exp\left(\frac{\beta_1 I_{pi}^T T_{pj}}{\tau}\right)}{\sum_{k \neq i} \exp\left(\frac{\beta_1 I_{pi}^T T_{pk}}{\tau}\right)}\end{aligned}$$

And vice versa for $\mathcal{L}^{t \rightarrow v}$

Since the weight matrix $\mathcal{W}$'s leading diagonal ($i = j$) is not included in the second summation, we can manually set it to -1 . This will combine the 2 summations into 1, which we can begin to vectorise.

$$\mathcal{L}^{v \rightarrow t} = \sum_{i=1}^B \log \sum_{j=1}^B \exp\left(\frac{I_{pi}^T T_{pj}}{\tau}\right) \mathcal{W}_{I_{pi} T_{pj}}^{v \rightarrow t}$$

Where $\mathcal{W}_{ii} = -1 \forall i$. We can set this manually.

Let $S_{ij} = \frac{I_{pi}^T T_{pj}}{\tau}$ where $I \in \mathbb{R}^{B \times n}, T \in \mathbb{R}^{B \times n}$ are the batch matrices of each image and text vector row-by-row.

So $S = \frac{1}{\tau} I T^T \in \mathbb{R}^{B \times B}$ is a similarity matrix.

We normalise I and T to prevent gradient blow-up.

Next we define $A = \exp(S)$ element-wise.

So

$$\mathcal{L}^{v \rightarrow t} = \sum_{i=1}^B \log \sum_{j=1}^B A \mathcal{W}$$

after setting $\text{diag}(\mathcal{W}) = -1$

Note that in order to calculate $\mathcal{L}^{t \rightarrow v}$ we require $T_{pi}^T S_{pj}$, which is just S^T .

$$\mathcal{W} = (B - 1) \frac{A_{ij}^{\beta_1}}{\sum_{k \neq i} A_{ij}^{\beta_1}} = (B - 1) \frac{e^{\beta_1 S_{ij}}}{\sum_{k \neq i} e^{\beta_1 S_{ik}}}$$

We can define $X = \beta_1 S$ element-wise and set $X_{ii} = -\infty$

Then $\mathcal{W}^{v \rightarrow t} = (B - 1) \cdot \text{softmax}(X, \text{dim} = -1)$ computationally

We would transpose X to find $\mathcal{W}^{t \rightarrow v}$

So our final vectorised algorithm looks like:

1. Calculate $S = \frac{1}{\tau} I T^T$
2. Calculate $A = \exp(S)$ element-wise
3. Calculate $X = \beta_1 S$ and then set the leading diagonal to $-\infty$
4. Calculate $\mathcal{W} = (B - 1) \cdot \text{softmax}(X, \text{dim} = -1)$
5. Set the leading diagonal of $\mathcal{W}$ to -1
6. $\mathcal{L}^{v \rightarrow t} = \sum(\log(\sum(A * \mathcal{W})))$
7. Calculate $\mathcal{L}^{t \rightarrow v}$ by repeating steps 4-6 with X^T (to apply softmax to the other axis)

$$\mathcal{L} = \mathcal{L}^{t \rightarrow v} + \mathcal{L}^{v \rightarrow t}$$